

Convolutional Neural Network

Gabriel D'Assumpção de Carvalho

`gdc2@cin.ufpe.br`

Ciências Atuariais - UFPE

11 de junho de 2025

Programação

- 1 Regressão Logística
 - Distribuição de Bernoulli
 - Estimação dos Parâmetros
- 2 Redes Neurais Convolucionais
 - Convolução
 - Tensores
 - Max-Polling
 - Comprimir Canais
 - Estruturas de CNN Modernas
- 3 Redes Generativas
 - Autoencoders (AEs)
 - Variational Autoencoders (VAEs)
 - Generative Adversarial Networks (GANs)
- 4 U-net

Regressão Logística

Introdução

Na teoria das probabilidades e estatística, a **distribuição de Bernoulli** é uma distribuição discreta com espaço amostral $\{0, 1\}$, nomeada em homenagem ao matemático suíço Jakob Bernoulli.

Ela modela experimentos com dois resultados possíveis:

$Y = 1$ com probabilidade de **sucesso** p ,

$Y = 0$ com probabilidade de **fracasso** $q = 1 - p$.

Formalmente:

$$Y_i \sim \text{Bernoulli}(p_i), \quad P(Y_i = 1) = p_i, \quad P(Y_i = 0) = 1 - p_i.$$

Quais Variáveis Seguem Distribuição de Bernoulli?

Considere as variáveis abaixo. Marque **V (Verdadeiro)** se a variável pode ser modelada diretamente por uma **distribuição de Bernoulli** (i.e., variável binária com dois possíveis resultados, como 0 ou 1). Marque **F (Falso)** caso contrário e indique como a variável poderia ser transformada para se adequar a esse modelo.

- () Uma lâmpada está queimada ou funcionando.
- () Número de chamadas recebidas em um call center por hora.
- () Um cliente realizou uma compra durante uma visita.
- () Cor de um carro (vermelho, azul, preto, branco).
- () Aluno aprovado ou reprovado em uma disciplina.
- () Quantidade de gols marcados em uma partida.
- () Segmentação semântica de imagens, onde cada pixel é classificado como pertencente ou não a um objeto específico (exemplo: carro).
- () Durante o dropout, a ativação da camada inteira (ou de um subconjunto de neurônios) é determinada por uma única variável aleatória com distribuição de bernoulli.

Discussão: Quais variáveis podem ser tratadas como binárias com reclassificação ou simplificação? Como isso impacta o modelo?

Propriedades (Parte 1)

Em modelos lineares generalizados (GLM), modelamos a esperança condicional:

$$\mathbb{E}[Y_i | X_i] = p_i = f(X_i^\top \beta)$$

Como $X_i^\top \beta \in \mathbb{R}$ e $p_i \in [0, 1]$, usamos uma função $f : \mathbb{R} \rightarrow [0, 1]$.

Principais propriedades:

Função de probabilidade da Bernoulli:

$$f(Y_i; p_i) = p_i^{Y_i} (1 - p_i)^{1 - Y_i}, \quad Y_i \in \{0, 1\}.$$

Esperança:

$$\mathbb{E}[Y_i] = p_i.$$

Função de ligação (logit):

$$\ln \left(\frac{p_i}{1 - p_i} \right) = X_i^\top \beta.$$

Propriedades (Parte 2)

Função verossimilhança:

$$L(\beta) = \prod_{i=1}^n p_i^{Y_i} (1 - p_i)^{1 - Y_i}$$

Log-verossimilhança:

$$\ell(\beta) = \sum_{i=1}^n [Y_i \log(p_i) + (1 - Y_i) \log(1 - p_i)]$$

Encontre p_i e substitua em $\ell(\beta)$.

Estimação via Gradiente

Para maximizar $\ell(\beta)$, usamos o gradiente descendente. A função **sigmoide** é:

$$p_i = \frac{1}{1 + e^{-z_i}}, \quad z_i = X_i^\top \beta$$

A derivada da log-verossimilhança, via regra da cadeia:

$$\frac{\partial \ell}{\partial \beta} = \frac{\partial \ell}{\partial p_i} \cdot \frac{\partial p_i}{\partial z_i} \cdot \frac{\partial z_i}{\partial \beta}$$

Onde:

$$\ell = y_i \log(p_i) + (1 - y_i) \log(1 - p_i)$$

$$p_i = \frac{1}{1 + e^{-z_i}}$$

$$z_i = X_i^\top \beta$$

Derive cada termo para obter a expressão final do gradiente e atualizar β .

Redes Neurais Convolucionais

Operação de Convolução

Convolução é um operador linear que mede a sobreposição entre uma função de entrada e um filtro (kernel).

Definições matemáticas:

Convolução discreta:

$$(f * g)(x) = \sum_{k=-\infty}^{\infty} f(k) \cdot g(x - k)$$

Convolução contínua:

$$(f * g)(x) = \int_{-\infty}^{\infty} f(k) \cdot g(x - k) dk$$

Em redes neurais, aplica-se essa operação sobre imagens usando pequenos filtros que se deslocam para capturar padrões locais.

Exemplo Probabilístico

Convolução em distribuições de probabilidade:

Suponha X_1 e X_2 como as faces obtidas em dois lançamentos de dados. A probabilidade da soma assumir determinado valor pode ser obtida por convolução:

$$P(X_1 + X_2 = 4) = (P_{X_1} * P_{X_2})(4)$$

x	1	2	3	4	5	6
$P(X = x)$	$\frac{1}{6}$	$\frac{1}{6}$	$\frac{1}{6}$	$\frac{1}{6}$	$\frac{1}{6}$	$\frac{1}{6}$

Tabela: Distribuição uniforme de um dado

Tensores em Visão Computacional

Representações em Álgebra Linear:

Escalar: tensor 0D

Vetor: tensor 1D

Matriz: tensor 2D

Imagem RGB: tensor 3D

Quando a imagem é 2D ou 3D?

2D: imagem em tons de cinza ($H \times W$)

3D: imagem colorida com canais ($H \times W \times C$)

Exemplo de Convolução

Dados:

Imagem de entrada: 3×3

Filtro: 2×2

Stride = 1

Padding = 0

Pergunta: Qual a dimensão da saída?

$$\text{Output} = \left\lfloor \frac{(N - F + 2P)}{S} + 1 \right\rfloor$$

E se o padding for 1?

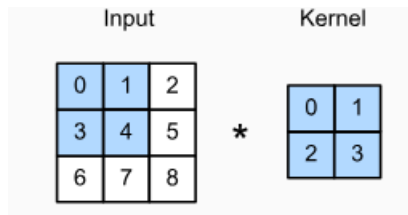


Figura: Convolução Matriz 3x3 e Filtro 2x2

Correlação Cruzada com Múltiplos Canais

Quando lidamos com imagens ou dados com múltiplos canais (por exemplo, RGB), tanto a entrada quanto o kernel (filtro) são tensores com múltiplas "fatias", uma por canal.

Processo:

Para cada canal da entrada, aplicamos a correlação cruzada com o kernel correspondente.

O resultado de cada canal é um mapa 2D (saída parcial).

A saída final é a **soma ponto a ponto** de todos os mapas parciais.

Resumo visual: soma das correlações por canal:

$$Y = \sum_{c=1}^C (X_c \star K_c)$$

Exercício: Calcule o output da correlação cruzada mostrada na figura abaixo.

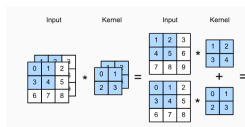


Figura: Cálculo de correlação cruzada com dois canais de entrada.

Max-Pooling e Redução de Parâmetros

Contexto:

Imagem de entrada: $32 \times 32 \times 3$

Após `flatten`: vetor com 3072 elementos

Conectada a uma camada densa com 20 neurônios

Perguntas:

Quantos parâmetros são gerados nesta camada totalmente conectada?

Como o uso de max-pooling pode reduzir esse número?

Exemplo de Max-Pooling:

Aplique uma janela 2×2 com *stride* = 1 na matriz abaixo.

0	1	2
3	4	5
6	7	8

Figura: Matriz 3×3 para operação de Max-Pooling

Convolução 1×1

Questões para discussão:

O que exatamente uma convolução 1×1 faz em um tensor?

Por que utilizar esse tipo de convolução em vez de filtros maiores?

Como a convolução 1×1 atua sobre os canais da entrada?

Qual o efeito dessa operação sobre a dimensão do tensor?

Qual arquitetura propôs esse tipo de operação? Em quais outras ela foi utilizada?

Exemplo:

Entrada: tensor de dimensão $(32 \times 32 \times 125)$

Aplicação: convolução 1×1 com 64 filtros

Pergunta: qual será a nova dimensão da saída?

Quem sou eu? Arquiteturas Modernas de CNN

Responda às perguntas identificando a arquitetura de rede convolucional descrita:

1. Fiquei famoso por fazer referência a um filme de mesmo nome. Minha arquitetura usa múltiplos filtros em paralelo com diferentes tamanhos em um mesmo bloco.



Figura: A origem (filme)

Quem sou eu? Arquiteturas Modernas de CNN

2. Fui pioneira ao ser treinada em GPUs. Mostrei as limitações da sigmoide/tanh e introduzi técnicas como ReLU, dropout e batch normalization. Quem sou eu? Quais problemas essas técnicas resolveram?
3. Padronizei blocos com convolução, ativação e pooling, facilitando o design de CNNs mais profundas sem perda de dimensão. Quem sou eu?
4. Permiti o aprendizado de funções identidade usando conexões de atalho (skip connections), facilitando a retropropagação em redes profundas. Quem sou eu?

Redes Generativas

O que são Redes Generativas?

As redes neurais generativas são modelos capazes de aprender a distribuição dos dados de entrada e gerar novas observações semelhantes.

Ideia central: Existe uma estrutura latente oculta que explica a variabilidade dos dados. Em visão computacional, isso pode representar:

- Iluminação da cena,
- Posição ou proporção de objetos,
- Variações de cor, ângulo ou ruído.

Perguntas para reflexão:

- O que significa "aprender a distribuição dos dados"?
- Como um modelo pode gerar novas amostras nunca vistas?

Autoencoders

Autoencoders são redes neurais treinadas para reproduzir sua entrada na saída, passando por uma representação intermediária de menor dimensão.

Estrutura:

Codificador: $X \rightarrow Z$ (redução dimensional)

Decodificador: $Z \rightarrow \hat{X}$ (reconstrução)

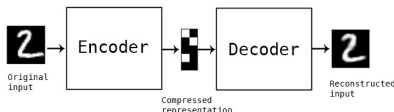


Figura: Arquitetura de um Autoencoder

Perguntas para discussão:

Qual função de perda usamos para treinar um Autoencoder?

Em que sentido ele é similar a análise de componente principal (pca)?

Por que queremos reduzir a dimensão dos dados?

VAEs: Autoencoders com Variabilidade

VAEs adicionam aleatoriedade à camada latente: em vez de mapear $X \rightarrow Z$ diretamente, o modelo aprende uma distribuição $q_{\theta}(Z|X)$ e amostra diferentes Z a cada vez.

$$X \rightarrow (\mu, \sigma^2) \rightarrow Z \sim N(\mu, \sigma^2) \rightarrow \hat{X}$$

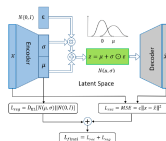


Figura: Arquitetura de um VAE

Perguntas para discussão:

Qual o papel da distribuição $q_{\theta}(Z|X)$?

Por que assumimos que $p(Z) \sim N(0, 1)$?

O que representa o termo de regularização na perda dos VAEs?

Como o modelo é capaz de gerar imagens diferentes para um mesmo input?

GANs: Redes Gerativas Adversariais

As GANs consistem em dois modelos treinados de forma competitiva:

Gerador (G): recebe um vetor de ruído $z \sim \mathcal{N}(0, I)$ e gera uma amostra sintética $G(z)$.

Discriminador (D): recebe uma amostra (real x ou gerada $G(z)$) e tenta classificá-la como real (1) ou falsa (0).

Objetivo: G tenta enganar D gerando amostras indistinguíveis das reais. D tenta acertar a classificação real/falso.

Perguntas para discussão:

Qual é a função de perda que o discriminador minimiza?

O gerador tenta maximizar a perda da cross-entropy para qual rótulo?

O que significa o jogo de min-max entre G e D?

Funções de Otimizações em GANs

Função do Discriminador (D):

$$\min_D \left[-y \log D(x) - (1 - y) \log(1 - D(x)) \right]$$

$y = 1$ para amostras reais, $y = 0$ para amostras falsas.

D tenta maximizar a verossimilhança de classificar corretamente.

Função do Gerador (G) – Versão Original:

$$\max_G \left[\log(1 - D(G(z))) \right]$$

Função Alternativa para o Gerador:

$$\min_G \left[-\log D(G(z)) \right]$$

Forma Conjunta (Jogo Min-Max):

$$\min_G \max_D \left\{ \mathbb{E}_{x \sim p_{\text{data}}} [\log D(x)] + \mathbb{E}_{z \sim p_z} [\log(1 - D(G(z)))] \right\}$$

Referência: <https://arxiv.org/pdf/1406.2661>

Exemplo: Análise das Funções de Perda do Gerador

Discussão:

Por que a versão original da perda do gerador, baseada em $\log(1 - D(G(z)))$, pode gerar gradientes fracos quando D está muito certo?

Por que a versão alternativa, baseada em $-\log(D(G(z)))$, é preferida na prática?

Complete a tabela a seguir com os limites:

$$\lim_{D(G(z)) \rightarrow \{0,1\}} \log(1 - D(G(z))) \quad \text{e} \quad -\log(D(G(z)))$$

Situação	$D(G(z))$	$\log(1 - D(G(z)))$	$-\log(D(G(z)))$
G ruim (D acerta)	$\rightarrow ?$	$\rightarrow ?$	$\rightarrow ?$
G bom (D erra)	$\rightarrow ?$	$\rightarrow ?$	$\rightarrow ?$

Tabela: Comparação dos comportamentos das perdas do gerador

U-net

U-Net

Referência: <https://arxiv.org/pdf/1505.04597>

Perguntas para discussão:

Qual é a função de erro minimizada pela U-Net?

Em quais tipos de problemas ela pode ser aplicada?

Por que essa arquitetura é chamada de “U”?

Qual é o papel das conexões entre as camadas de downsampling e upsampling?

Quais as vantagens dessas conexões em tarefas de segmentação?

Equivalência Softmax e Sigmoide para 2 Classes

Softmax com 2 classes calcula as probabilidades de cada classe $k \in \{0, 1\}$ como:

$$P(Y = k | x) = \frac{e^{z_k}}{e^{z_0} + e^{z_1}}$$

Para duas classes, isso é equivalente a aplicar a **função sigmoide** em um único logit $z = z_1 - z_0$:

$$P(Y = 1 | x) = \sigma(z) = \frac{1}{1 + e^{-z}}$$

Assim, a saída da sigmoide representa a probabilidade da classe 1, enquanto a classe 0 tem probabilidade complementar $1 - \sigma(z)$.

Portanto, **a sigmoide modela uma distribuição Bernoulli**, e a softmax com duas classes é uma generalização que, neste caso, colapsa para essa função.